ChatGPT

Executive Summary

PotFoundry's web geometry engine uses a hybrid CPU/GPU pipeline to generate watertight 3D pot meshes with decorative surface patterns. A CPU **meshBuilder** constructs a base mesh (outer/inner walls, rim, bottom, drain) from parametric dimensions and style functions, while a WebGPU-based **AdaptiveExportComputer** refines the mesh adaptively for high fidelity. We identified three primary failure modes:

- **Surface Fidelity Mismatch:** The exported mesh deviates from the intended mathematical surface (ridges not sharp, patterns "drift" or misalign). Root causes include misapplication of the twist rotation, incomplete enforcement of pattern features as mesh constraints, and omission of base profile curvature in refinement criteria. For example, earlier logic applied the style modulation at a rotated angle, causing pattern misalignment. This was corrected by evaluating style at the original angle and then rotating coordinates by the twist [1] [2]. However, the adaptive tessellation still does not strictly enforce sharp ridges as continuous edges – it "snaps" new vertices toward feature points, but the features are not embedded as actual constraint edges, leading to residual drift and smoothing of intended ridges.

- **Mesh Triangle Explosion:** Exports sometimes reach 40+ million triangles (~2 GB STL) under moderate settings. The GPU refinement subdivides the mesh based on a curvature/error threshold but lacked an effective global budget cap. In the quad-based path, every quad with error above threshold keeps subdividing until a min-size limit or memory cap. We see no use of the `targetTriangles` parameter in the refine loop – subdivision continues until the **MAX_QUADS**/**MAX_TRIANGLES** buffers fill. In one case, the process hit the defined memory cap (~8 million quads), generating ~43 million triangles before stopping. The **targetTriangles** budget was not actively enforced in code, allowing runaway refinement. The new triangle-subdivision path does include a check and break when triangle count approaches a limit, but the default limits were set too high (e.g. 8M quads, 4M triangles) to prevent massive meshes. In effect, the adaptive algorithm over-refines high-frequency pattern details (e.g. 31 ripples × 7 petals in *HarmonicRipple* style) globally, instead of focusing where needed.

- **Intermittent Feature Detection Failure:** The GPU feature extraction shader (`feature_extract.wgsl`) sometimes outputs zero feature points, even when sharp pattern features exist. This occurs if the curvature-based criteria fails to mark ridges/valleys, often due to threshold tuning or sampling resolution. The shader samples the radius field on a fine grid and flags local curvature extrema above a threshold as ridges (featureType=1) or valleys (2). If the threshold is set too high or the pattern's contrast is mild, **no points are emitted** (the atomic counter remains 0) – a case we observed intermittently. In such cases the adaptive refinement lacks any feature guidance and may treat the surface as globally smooth. This leads to undershooting (ridges not refined at all) or overshooting (refining everywhere trying to satisfy a low error tolerance). The code confirms ridges are only added if curvature K_max exceeds a threshold and the point is a strict local extremum. Under certain parameter combinations, numerical precision or aggressive threshold values cause all features to be skipped. Moreover, the current feature extraction only targets radial

pattern curvatures; it does **not** tag the pot's sharp structural edges (the rim edge, bottom corner) as "features." Thus, without manual handling, those edges might not get special refinement, depending on the error metric.

Impact & Severity: These issues are significant. Fidelity problems result in visible artifacts – smoothed-out ridges, "wavy" or drifting patterns, and possibly slight seams – undermining design intent. The triangle-count explosion causes huge file sizes and performance problems in downstream tools, and in extreme cases could crash the browser/tab due to memory exhaustion. Feature detection failure introduces unpredictability (e.g. user sees a ridge in preview, but exported mesh lacks the sharp edge). Overall, reliability and performance of the export pipeline are compromised.

Probable Causes: We traced the fidelity issues primarily to **algorithmic handling of the twist and feature constraints**. The twist angle was initially applied inconsistently – the pattern function took `θ+twist` while geometry was rotated, effectively double-twisting the pattern or misaligning it. The latest code corrects this ("CRITICAL FIX" in `meshBuilder.ts` – style evaluated at original θ, then coordinates rotated by twist). After this fix, the pattern orientation is consistent (e.g. petal ridges now spiral correctly with the pot). Remaining fidelity errors stem from the **adaptive meshing strategy**: it does not explicitly enforce ridges as edges, but rather relies on high tessellation density and vertex snapping to approximate them. Since multiple triangles may approximate a ridge, slight offsets or a "zigzag" ridge line can occur instead of a perfectly smooth crest. Additionally, the refinement metric focuses on radial curvature (pattern detail) and ignores the pot's longitudinal curvature (the smooth flare). Thus, if the base mesh resolution is low, broad smooth surfaces might stay faceted (no further splits because "no high curvature" by the metric). We confirmed that `compute_importance` uses `compute_style_variation` for outer/inner walls but does **not** account for the curvature of the base profile (e.g. a strong flare exponent). This means large-scale shape curvature is only controlled by the initial grid (`n_theta`, `n_z`), not the adaptive pass – a potential fidelity issue for smooth silhouettes.

The triangle explosion is rooted in the **refinement loop criteria**. Every quad/triangle subdivides if "importance" > threshold and size > min. With high-frequency styles, *every* cell exceeds the threshold at first, leading to exponential mesh growth. The intended safeguards (e.g. `maxDepth`, `targetTriangles`) were either not wired in or not tuned. We see that `targetTriangles` is set in the uniforms but never actually used to terminate or coarsen refinement. The only halting conditions in triangle mode are when no new triangles are generated in an iteration or when hitting 95% of `MAX_TRIANGLES` capacity. In practice, with complex styles the algorithm hits the cap rather than naturally converging, hence the 43M triangle outputs. In one log, the system did warn "'Max triangles reached during subdivision.'" before finalizing the mesh – indicating it stopped only due to the hard limit, not because the surface was sufficiently approximated.

Feature extraction's sporadic failures appear to be a combination of **sensitivity and continuity issues**. The ridge detector uses second-difference of radii on the θ–z grid and applies *non-maximum suppression* (each candidate must be higher than its neighbors). If a ridge is even slightly flattened or if the pattern amplitude varies, some ridges might not register above the fixed threshold. Also, ridges that move (twist) with height might not produce a continuous "feature line" in the slice-by-slice detection, causing fragmentation that could be filtered out. We did not find explicit clustering of feature points by length in the shader (the `minFeatureLen` from uniforms suggests an intent to filter short features, but that logic was not clearly present in the WGSL code). It's likely done on the CPU after reading back the features. A too-aggressive filter could accidentally drop all features in some cases, or if the GPU grid resolution or floating precision causes missed extrema at exact integer grid points (e.g. a ridge peak lying between sample points). The result is an

empty feature list. The code doesn't gracefully handle a no-feature scenario – it proceeds with pure error-based refinement, which can either over-tessellate or under-tessellate as noted.

In summary, the PotFoundry-Lite geometry engine's design is ambitious, but these issues in twist handling, feature constraint enforcement, and adaptive refinement heuristics are causing the fidelity and performance problems. We recommend a combination of tactical fixes (to salvage the current approach) and possibly a redesigned meshing strategy to robustly handle sharp features and limit mesh density. Below we map the system architecture and pinpoint specific problem areas, followed by a proposed plan for improvement.

Architecture Overview (Pipeline & Components)

Core Geometry Definition: The pot's surface is defined parametrically. The base profile is a revolution from z=0 (bottom) to z=H (height). At each z, a **base radius** is computed as a smooth interpolation between bottom radius Rb and top radius Rt (with an exponential flare factor). This base radius can include a "bell" bulge for mid-height shaping. On top of this, an optional **style function** modulates the radius as a function of angle θ (0 to 2π) and height. For example, the *HarmonicRipple* style adds petal-like sinusoids in θ and small ripples that vary with z [3]. The **surface point** P(θ, z) is thus defined in cylindrical coordinates by:

- Radius = r_outer_fn(θ, z) = baseRadius(z) * f_style(θ, z)
- Position = (x, y, z) with x = radius * cos(θ_total), y = radius * sin(θ_total)

Here θ_total may include an accumulated twist rotation along z. The twist (if any) is specified by parameters (spin_turns, spin_phase) meaning the pattern rotates as we go up, creating helical effects. The coordinate systems must remain consistent so that the pattern "sticks" to the pot as it twists.

CPU Mesh Generation (meshBuilder.ts & related): The CPU path (`buildPotMesh()` in **meshBuilder.ts**) constructs a watertight triangle mesh by sampling the parametric surface on a fixed grid (n_theta around, n_z vertically). It mirrors the original Python implementation (`geometry.py`). It generates vertices for: outer wall, inner wall (offset inward by wall thickness), bottom (both underside and top inner bottom), rim (connecting outer and inner at the top), and the drain hole cylinder. Each section's vertices are indexed into triangles in a consistent winding (ensuring each quad of the parametric grid yields two triangles). The CPU mesh builder ensures **seam continuity** (θ=0 and θ=2π join) by using `endpoint=False` on the theta linspace – the last segment wraps around and indices are connected modulo n_theta [4] [5]. Likewise, outer and inner walls share the top rim ring so that the rim triangles stitch them together [6]. The code also clamps inner radii near the drain to avoid negative thickness (ensuring watertightness around the drain hole) [7] [8].

One key aspect is **twist consistency**. In the current implementation, the CPU builder applies twist by rotating the coordinates of each vertex, rather than feeding the twist into the radius function. This is evident in `geometry.py` where `r_outer_fn` is called with the original theta (no twist added) and then the (x,y) is rotated by the twist angle [9]. The comment "no + twist here" confirms the convention [10]. This approach matches the corrected TS implementation: `styleFn(thetas, z, r0, …)` is evaluated with original angles, and then each ring of vertices is rotated by the computed twist. This ensures a pattern (say a petal at θ=0) will move to a new physical angle as z increases, but the mathematical evaluation of the petal shape is in its local frame. The result is the pattern "twists with" the pot with no distortion. (Previously, passing

θ+twist into the style function *and* rotating coordinates effectively over-rotated the pattern relative to the pot.)

GPU Adaptive Refinement (AdaptiveExportComputer.ts & shaders): For high-detail export, the system offloads mesh refinement to WebGPU compute shaders. There are two modes:

- **Full Parametric Tesselation Mode:** (Original design) The GPU starts with a coarse grid of quads in parametric (θ, t) space (e.g. 360 segments in θ by some in t). The shader `init_coarse_grid` seeds an array of quads covering all surfaces (outer, inner, rim, etc., encoded with surface IDs). A loop then subdivides quads based on an error metric. Each quad's "importance" is evaluated via `compute_importance` which looks at pattern variation and surface curvature within that quad. If importance > threshold, the quad is split into 4 smaller quads (midpoints are inserted in θ and t). This process can repeat iteratively (though in the WGSL it's unrolled or repeated via multiple dispatches). Finally, when quads are below the threshold or a max depth, any remaining quads are output as triangles. Vertex positions are then evaluated for all triangle vertices using the parametric equations (in shader, e.g. `compute_outer_radius` and `compute_twist`) and written to the mesh buffers.

- **Base Mesh Subdivision Mode:** (Refactored approach) The CPU-generated mesh (or any provided base mesh) is uploaded and used as a starting point. The base mesh's triangles are then subdivided on the GPU to add detail. This mode is activated when `params.baseMesh` is provided. The base mesh's vertices and indices are loaded into GPU buffers, and an initial triangle list is prepared (each triangle knows which surface it belongs to). A compute pass `subdivide_triangles` then processes each triangle: it can insert new vertices at mid-edges (or other strategic points) and produce up to 4 new smaller triangles (classic triangle bisection refinement). The criteria is similar – an estimate of error (e.g. deviation of the surface from flat across the triangle) is compared to `subdivThreshold`. The code snippet shows this process: each iteration, `subdivide_triangles` writes a new list of triangles to a "next" buffer and the count is checked. The loop runs until either no new triangles are added (convergence) or a max depth or triangle cap is reached. Finally, a pass `emit_final_triangles` outputs the subdivided triangles, and `evaluate_vertices` computes the final vertex coordinates (mapping parametric (θ,t) stored in each vertex to XYZ positions).

Both modes use the **feature extraction** results to preserve important edges. The feature points (angle θ_f and height parameter t_f for ridges/valleys) are precomputed (on GPU) and passed in a buffer to the refinement shader. In the quad mode, the `compute_importance` uses features implicitly by boosting the error near features (though the code we reviewed suggests this was planned but not fully implemented, aside from snapping). In the triangle mode, a critical step is **feature snapping**: when emitting new vertices, the shader adjusts a vertex's parametric coordinates to snap exactly to a nearby feature point if one lies within a certain radius. For example, if a new vertex is being placed along a ridge line (feature), its θ,t is moved to coincide with that ridge. This effectively ensures the ridge becomes a line of collinear vertices in the mesh. However, if the base mesh never had an edge along that ridge, the snapping will create a chain of vertices that lie on the ridge but are connected in a zigzag fashion (still an approximation). The system relies on refining enough that those zigzags flatten out into an apparent continuous edge.

Export and Post-processing: Once the refined mesh is computed, it is either directly downloaded as STL (the code uses a WASM/JS STL writer). The design mentions possibly using **meshoptimizer** to decimate the mesh after generation (to reduce triangle count), and supporting 3MF export (`export3MF.ts`) for color/

units preservation. The current pipeline, however, outputs STL (likely binary STL given the .zip's context and the evolution plan) by default. No evidence of an active decimation step was seen in the provided code; if implemented, it would use meshoptimizer to simplify the mesh while preserving geometry (likely as an optional step or quality setting).

Summary of Data Flow:

1. **Input:** Pot dimensions (height, diameters, thickness, etc.), style selection and its parameters, and quality settings (`n_theta, n_z` for base mesh; optional target triangle count or threshold).
2. **CPU Generation:** Build coarse mesh (if used for preview or as base for GPU).
3. **GPU Feature Extraction:** Sample parametric surface at high resolution; detect curvature extrema as features (list of (θ, t) + type).
4. **GPU Adaptive Refinement:** Either generate initial parametric grid or upload base mesh. Iteratively subdivide, referencing **uniforms** (dimensions, twist, style parameters) and **feature_buffer**:
5. Calculate error importance for each cell/triangle (higher near features or high curvature).
6. Subdivide if error > threshold, until limits.
7. Snap new vertices to feature lines (θ,t adjusted).
8. Output final triangles.
9. Compute final vertex coordinates in world space.
10. **Output Mesh:** Triangles (faces) and vertices are collected; the STL exporter writes them out. Diagnostics like triangle count, clamped inner wall count, etc., are gathered (e.g., `clamp_count` for inner wall appears in diagnostics).

This architecture is powerful – it aims to concentrate detail where needed (sharp pattern features) while keeping smoother areas coarse. It ensures watertightness by construction (the base mesh and subsequent subdivisions are careful to always subdivide consistently across shared edges). However, as we've seen, certain bugs in each stage undermine its effectiveness. Below is a **Bug & Risk Register** summarizing these issues, with references to code and recommended fixes:

Bug & Risk Register

Issue & Impact	Location (File : Line)	Severity	Proposed Fix
Twist misapplication causing pattern drift: Initially, the pattern function was evaluated at θ+twist while also rotating coordinates by twist, effectively double-twisting or misaligning patterns (ridges didn't follow the pot's twist exactly). This led to subtle distortion of features (petals "drifting" around the pot).	`meshBuilder.ts` (see logic in `buildPotMesh`), also `geometry.py` [9] [11]	Medium	**Resolved in code** by evaluating style with original θ and applying twist only in coordinates. Ensure all generators (CPU and GPU) follow this convention. Double-check GPU's `compute_twist` vs `compute_outer_radius` usage in shader: the snippet for vertex eval shows using `compute_outer_radius(θ, t)` and then `compute_twist(θ, t)` for coordinates – this should mirror the CPU logic. Consistency here fixes the drift.
Base profile curvature not in refinement metric: The adaptive criterion ignores the pot's smooth shape curvature (flared profile), so broad curves may remain faceted if base mesh was coarse. Only style-induced curvature triggers splits.	`adaptive_mesh.wgsl` `compute_importance()`	Medium	Incorporate a measure of curvature in the z-direction. For example, compare the actual surface vs linear interpolation across a quad's vertical span. In shader, we can sample `compute_outer_radius` at quad corners and midpoints in *height* to capture vertical deviation. Add that to `base_imp`. Alternatively, ensure `n_z` in base mesh is high enough or allow a low threshold to also subdivide long quads even if pattern is smooth.

Issue & Impact	Location (File : Line)	Severity	Proposed Fix
Over-aggressive refinement (triangle explosion): The refinement loop subdivides virtually everything for complex styles. No effective stop until hitting memory cap, yielding 43M triangles in one case. This is due to the static `subdivThreshold` and lack of early exit criteria tied to a triangle budget.	`adaptive_mesh.wgsl` (quad mode); `AdaptiveExportComputer.ts` (triangle mode loop)	**High**	Implement **triangle budget enforcement** and adaptive thresholding: e.g. if `targetTriangles` is set (say 1e6), dynamically raise the error threshold once that count is approached to stop further splits. In triangle mode, use `params.targetTriangles` – if next iteration would exceed it, stop. In quad mode (if still used), similarly break out of the refinement loop by checking atomic counters against the budget. Additionally, tune `subdivThreshold` per style frequency: extremely detailed styles could use a higher threshold by default to prevent runaway.

Issue & Impact	Location (File : Line)	Severity	Proposed Fix
Feature extraction missing features or returning none: Occasionally no ridges are detected (features list empty) even when pattern has pronounced features, leading to either under-refinement (if threshold high) or uniform over-refinement (if threshold low). Also, structural sharp edges (rim, bottom) aren't marked as features by the current shader.	`feature_extract.wgsl`	High	Adjust the feature detection algorithm: (1) Lower the sensitivity threshold or make it scale with pattern amplitude (e.g. compute global max curvature first, set threshold as a fraction of it). (2) Ensure a minimum set of features: always include the bottom corner and rim as "crease" features (featureType=3). These can be added in CPU code using known geometry (θ any, t=0 for bottom edge, t=1 for rim). (3) Implement continuity filtering carefully – instead of dropping short segments completely, perhaps reduce their strength but keep at least a few points so snapping has something to lock onto. Also consider increasing grid resolution for feature extract (currently likely 2048×1024); if performance allows, a finer grid reduces the chance of missing a narrow peak.

Issue & Impact	Location (File : Line)	Severity	Proposed Fix
Feature constraints not enforced as edges: The current pipeline only *snaps* vertices to features; it doesn't guarantee a continuous edge in the mesh. This can lead to ridges being approximated by a jagged line of triangles rather than a clean edge, especially if base mesh didn't align with the feature.	`adaptive_mesh.wgsl` vertex emission	Medium	Use **constrained triangulation** so that feature lines appear as actual mesh edges. One approach: project feature lines into (θ,t) domain and split the initial mesh along those lines *before* refinement. For example, insert a row of vertices along a detected ridge and split adjacent faces. This turns the ridge into a boundary in the parametric grid so refinement will naturally keep it sharp. This could be done on CPU after feature detection (using a constrained Delaunay triangulation in (θ,t) space including feature polylines). If full integration is complex, a simpler partial fix is to weight the error metric heavily along feature lines to force many skinny splits that effectively align to the ridge. This, however, increases triangle count, so the constrained edge approach is preferable for clean, low-count representation.

Issue & Impact	Location (File : Line)	Severity	Proposed Fix
Seam alignment and pattern periodicity: Potential minor issue – if a style function doesn't naturally repeat exactly at 2π (e.g. superformula might not unless parameters chosen carefully), a visible seam could appear. A `seamAngle` adjustment is passed in styleOpts (seen in code) to phase-shift patterns, but any mis-tuning would leave a small gap or misaligned feature at the seam.	`meshBuilder.ts` (injecting seamAngle)	Low	Test all styles for continuity at θ=0/2π. If issues, compute a phase offset that equalizes the radius at 0 and 2π. In styles that have an integer number of repeats (petals etc.), ensure that number * n_petals = 2π. The engine already exposes `qual.seamAngle` – verify each style uses it if needed (for instance, superformula or others might use seamAngle to rotate the pattern half a step to close smoothly). This is more a calibration than a code bug.
Inner wall self-intersection ("swallowtail") fix effects: The engine applies a Laplacian smoothing to inner wall radii after generating the mesh to avoid self-intersections in deep patterned valleys. While necessary, this could slightly reduce inner detail fidelity or wall thickness consistency.	`meshBuilder.ts` (inner wall smoothing section)	Low	This is a trade-off fix. Document that the inner surface may be slightly smoothed relative to the outer pattern to ensure printability. If needed, make the smoothing strength configurable or apply it only when a problematic geometry is detected (e.g. use diagnostics: if many inner points were clamped or if any triangle in inner mesh has a very acute angle).

(Table legend: High = must-fix for core functionality, Medium = significant impact but localized, Low = minor or cosmetic issue.)

Triangle Count Explosion – Root Cause Analysis

The extreme triangle counts are primarily due to **unbounded adaptive refinement** driven by fine pattern detail. In one scenario, a pot with **HarmonicRipple** style (7 petals, 31 ripples) was exported at "high" quality. The base mesh might start with n_theta=300, n_z=160 (~96k faces). The adaptive process then attempted to resolve the small ripples: each base quad in the outer wall contained about half a wavelength of the ripple pattern, so error was high everywhere. The refinement split each quad, then each of those, and so on. **Exponentially** many tiny triangles resulted, especially along ridges and in the bottom region where the pattern converges (e.g., ripples bunching toward the base).

Looking at the code, there was no mechanism to stop subdividing when an overall triangle budget was exceeded or when additional splits provided negligible improvement. The condition was purely local: *if local error > threshold, subdivide*. With a uniformly low threshold, a high-frequency pattern can trigger maximal subdivision uniformly across a surface region. In the worst case, triangle count grows ~4^D (D = depth of subdivision) per initial cell. Indeed, logs show the process hitting the hardcoded maximum: *"Max triangles reached...during subdivision."* before termination. That run produced ~43 million triangles, which correlates with starting ~168×84 grid and refining ~5 levels deep uniformly.

Why didn't `targetTriangles` **prevent this?** The `AdaptiveExportParams` include an optional `targetTriangles`, but we found it wasn't actively used in the refinement logic. In the triangle subdivision loop, the code sets `MAX_TRIANGLES = 4_000_000` and checks if `currentTriCount >= 0.95 * MAX_TRIANGLES` to break. This is a fail-safe, not a dynamic target. The `targetTriangles` value (if provided by the user) is passed into the uniform buffer (chunk4.z) but the shader doesn't reference it for stopping criteria. Essentially, the adaptive refinement was running open-loop with a fixed geometric error threshold.

Additionally, the error threshold itself may have been set too tight for complex styles. If `subdivThreshold` is globally a small value (aiming for high fidelity everywhere), the algorithm will over-tessellate. An ideal strategy would increase the threshold once the triangle count is high, concentrating splits only where needed most. This feedback was not implemented.

Triangle Budget Failure Modes: Running out of buffer space or hitting the max triangle cap in mid-refinement can result in an incomplete refinement. The code attempts a graceful fallback: if a quad split would overflow the `quads_next` buffer, it emits the quad as triangles as-is (preventing holes). However, those triangles might still be above error threshold. So the mesh stops improving once the cap is reached. The user sees a huge file that *still* might not perfectly capture the surface (some ridges might be a bit faceted because we aborted mid-process). Conversely, if we hadn't had a cap, it could crash or hang the system.

We also note the **memory overhead**: 43M triangles mean ~130 million vertices (since it's mostly an open surface, though shared vertices somewhat less). The GPU had buffers allocated for up to 100M vertices, which explains how it could reach that range. This is an enormous memory usage (hundreds of MB on GPU, 2GB on disk). So the system as is can easily exceed practical limits if not controlled.

Solution Analysis: We need a way to curb this growth while preserving necessary detail:

- **Adaptive Error Threshold:** Instead of a fixed threshold, use an iterative strategy. For example, start with a moderate threshold. Subdivide to budget or convergence. If triangles < target and detail lacking, lower the threshold and refine more. Essentially, allocate triangle count where needed most (multi-pass refinement). This prevents "overspend" of triangles on very fine but visually minor details.

- **Direct Triangle Budget Use:** Alternatively, estimate how many triangles each level of refinement would add and stop when the next level would exceed the budget. The code can sum the potential splits (each triangle above threshold produces up to 3 new ones). If that sum + current count > target, either globally raise the threshold or selectively allow only the largest errors to split. This leads into more complex *adaptive* refinement (non-uniform threshold), which is more advanced but yields optimal distribution of triangles.

- **Post-Refinement Decimation:** The plan to use meshoptimizer suggests an offline decimation of an over-refined mesh. If our pipeline overshoots, we could run simplification to bring triangle count down. Meshoptimizer can collapse edges under a error tolerance. However, decimation after the fact can undermine the purpose of adaptive refinement (which is to place triangles optimally in the first place). It's a fallback: if we do generate 40M triangles, decimate 10× to ~4M while hoping to preserve ridges. This is not as efficient as preventing those extra triangles to begin with.

- **Style-specific LOD tweaks:** Recognizing that some styles (e.g. micro ripples) simply do not require capturing every oscillation at maximum resolution for printability – we could intentionally limit subdivisions for certain high-frequency components. For instance, the 31-frequency ripple might be filtered out beyond a certain level of detail (effectively treating it as a bump map if below printer resolution). This is a product decision: maybe provide a "maximum resolution" slider or detect when triangle density exceeds printer resolution (~0.1 mm facets maybe) and stop.

In summary, the root cause is a one-size-fits-all refinement threshold applied to potentially highly detailed patterns without a global check. The fix is to bring *global awareness* to the refinement: tie it to a triangle budget or diminishing returns criterion. The next section outlines a refactored approach to meshing that incorporates these ideas and addresses the fidelity issues simultaneously.

Fidelity Issues Diagnosis (Ridges, Valleys, Smooth Curvature)

Ridges & Valleys Not Sharp: Ideally, a ridge (peak in radius) should form a continuous sharp edge on the mesh where two smooth surfaces meet (like a crease). In the current output, ridges are approximated by a band of highly tessellated triangles rather than a single crease line. The root cause is that the mesh is not being *split along* the ridge. Instead, it's just refined around it. Vertices on the ridge might align, but the triangulation still cuts across the ridge line. This is inherent to using a regular grid plus adaptive splits without topological change: you can approximate a sharp feature by many small facets, but it's never truly sharp unless you allow the feature to be a mesh edge.

We see this in the absence of any code constructing feature-aligned edges. The feature points are used to snap vertex positions but not to reconfigure connectivity. Thus, the ridge "floats" through the mesh. Slight misalignment or zigzag can occur, especially if the base mesh was coarse. For example, suppose a petal

ridge goes from base to top. The base mesh might have a vertex on that ridge only at certain z slices; between those slices the ridge passes inside quads. The adaptive subdiv will insert new vertices closer to the ridge in those quads, but depending on the pattern path, those new vertices might alternate sides of the actual ridge line. This yields a **sawtooth** approximation of the ridge. Visually, this can appear as a slight wobble or a ridge that's not perfectly straight. It also means the ridge's **sharpness** (dihedral angle) is diluted – instead of a true crease, we have a band of triangles with gradually changing normals.

Seam Continuity: If not properly handled, the seam at θ=0 can show a mismatch. The code tries to avoid this – using identical formula for θ=0 and θ=2π rings and connecting indices. The styles are (mostly) periodic functions (cos/sin) so values match at 0/2π. One possible issue: if a style uses a phase that isn't an integer multiple of 2π over the pot's twist (e.g., a spiral ridge that doesn't end exactly at the same angle it started), there could be an offset at the seam. The **seamAngle** injection suggests they allow a manual phase tweak. In testing, a subtle seam might appear if, say, you had a half-petal leftover. According to the STYLES definitions we saw, most patterns are inherently periodic (the number of petals is integer, etc.), so the seam likely is fine. If any style wasn't, that's easily fixed by adjusting phase or making endpoint periodic in the style function itself.

Smooth Curvature vs Faceting: As mentioned, broad smooth areas (like an un-patterned section or a gentle bell curve) might show faceting if the resolution was not sufficient and no further refinement was triggered. This is a fidelity loss because the user expects a smooth curve. For instance, if a user sets a high flare exponent, the profile might curve strongly near the top. If n_z was low (say 50), that curve could be polygonal in cross-section. The adaptive refine doesn't notice because radially the surface is flat (no pattern), so importance is zero. The result: the exported pot has noticeable bands near the rim. This is a corner-case, since typically one would increase base quality for overall shape, but it is a limitation in the algorithm.

We can mitigate this by including a curvature check for the profile or by always refining a bit in z for high flare or bell values. Another approach: tie the adaptive threshold to estimated surface deviation. If a quad spans a region where the surface normal changes by more than, say, 5°, we refine it even if style variation is nil. The rim underside and inner bottom might also need special consideration – these are small surfaces that meet at sharp angles with walls. The current mesher already splits the rim into many small triangles (because it's essentially one ring of quads split by angle, which might be fine), and the bottom is flat so no issue there.

Visual Drift of Patterns: "Drift" likely refers to patterns not being exactly where expected. Before the twist fix, drift was obvious: e.g., a ridge might start at a certain angle at the base but end a few degrees off at the top because the pattern wasn't properly following the twist. After the fix, that specific drift is solved – patterns move up with the correct helical motion. However, there could be a *mesh approximation drift*: the triangulation might not precisely trace the path of a ridge. If feature points detection was sparse, the snapping might pull vertices to slightly incorrect positions. For example, if the feature extraction sampled a ridge at discrete heights, the actual ridge between those heights might curve slightly, but the snapping will lock intermediate vertices to the nearest sampled point, effectively "quantizing" the ridge path. This can cause a slight meander away from the true analytic ridge. The higher the feature sampling resolution, the less drift, so increasing that or doing some interpolation of feature lines would help.

To verify these issues, we can implement tests in the next section. But first, we propose an improved architecture that addresses these core problems.

Proposed Solution Architecture

To achieve both high fidelity and controlled triangle count, we recommend a two-pronged approach:

A. Enforce Feature Lines as Mesh Constraints (Architectural Correctness): Modify the pipeline to **explicitly insert ridges and valleys as edges** in the initial mesh. This fundamentally improves fidelity by aligning the mesh topology with the surface's "lines of significance." We can accomplish this as follows:

1. **Feature Line Extraction:** Instead of just point sampling, construct continuous feature polylines. Take the output of the feature shader (a set of points with (θ, t)). Cluster them by featureType and continuity: sort/group by θ, and follow them in the t (height) direction. Essentially, connect the dots to form ridge curves. This can be done in parameter space: because our parameter space is rectangular (θ wraps, t from 0 to 1), a ridge likely appears as a roughly vertical curve (petal ridge) or diagonal (spiral ridge). We can trace it by starting from a detected point and moving to neighboring θ,t cells looking for next point of same type. This yields a polyline for each ridge/valley.

2. **Constrained Triangulation:** Perform a triangulation of the parametric domain (θ–t) that includes those polylines as non-crossing constraints. A suitable algorithm is Constrained Delaunay Triangulation (CDT) in 2D. We have existing libraries in JS/WebAssembly, or we could implement a simpler incremental triangulator since our domain is not too large. The input to the CDT would be:

3. Boundary of domain (the rectangle, which is already implicitly handled by connecting θ=0 and θ=2π edges).
4. All feature polylines (each becomes a constraint that must appear as edges in the triangulation).
5. Perhaps additional guide lines if needed (like the rim circle at t=1 could be a constraint if we want it exact, but that's already a boundary between surfaces outer/inner).
6. Optionally initial mesh vertices as extra points (but CDT can generate its own points; we might just enforce corner points of surfaces).

The CDT will generate a triangle mesh where ridges and valleys are edges. This immediately solves ridge continuity and reduces drift: the mesh geometry along those edges is exactly the analytic feature (since we will compute vertex positions on those edges using the analytic function).

1. **Adaptive Refinement on CDT Mesh:** Now apply adaptive refinement on this initial constrained mesh. Because the key features are locked in, the refinement can focus purely on *smoothness*. We subdivide triangles based on curvature error as before, but we **do not** remove or move the constraint edges. Triangles are split either by inserting new vertices on existing edges or splitting through interior. Importantly, the feature edges remain edges (we can split them if needed to add more vertices along them, but they stay aligned).

This approach yields far fewer triangles for a given fidelity: a sharp ridge that previously took, say, 16 triangles across to approximate can now be 2 triangles meeting at an edge (the edge itself representing the ridge exactly). It addresses fidelity (ridges are crisp and correctly located) and helps performance (less over-refinement needed around features, since the feature itself is resolved by an edge).

B. Smarter Adaptive Refinement (Efficiency and Control): Improve the adaptive algorithm to respect budgets and capture smooth curvature:

1. **Multi-Stage Refinement with Budget:** Introduce a parameter `targetTriangles`. Start with the constrained mesh from (A) which might have, say, a few thousand triangles. If target is, e.g., 500k, we can do iterative refinement: Each iteration, compute error on all triangles. If current triangle count + potential new triangles <= target (with some margin), subdivide all triangles above threshold. If subdividing all would overshoot, use a partial refinement: sort triangles by error and subdivide the worst first until budget is reached. This requires a priority queue of triangles by error – which is easier to manage on CPU, but could be approximated on GPU by one pass to label which to split (e.g., using a higher threshold such that only a fraction splits).

Pseudocode (conceptual):

```
mesh = initial_CDT_mesh
while mesh.triangle_count < targetTriangles:
    errors = [estimate_error(tri) for tri in mesh.triangles]
    max_err = max(errors)
    if max_err < tolerance: break  # target quality achieved
    # Determine splits:
    if mesh.triangle_count + count_if(errors > subdivThreshold) <=
targetTriangles:
        threshold = subdivThreshold
    else:
        # find the N largest errors that can be split within budget
        sort_errors = sorted(errors, reverse=True)
        N = targetTriangles - mesh.triangle_count
        threshold = sort_errors[N-1]  # split all triangles with error >= this
    for each triangle with error >= threshold:
        subdivide_triangle(triangle)
    update(mesh)  # new triangles, recompute neighbor relations etc.
    # Optionally raise subdivThreshold gradually to avoid refining tiny details
globally
```

This ensures we never exceed targetTriangles. Triangles with error below threshold are left as is (their error is tolerable or we ran out of budget).

1. **Incorporate Profile Curvature in Error Metric:** Define the error of a triangle not just by pattern variation but by how well it approximates the surface. For example, we can sample the actual surface at a triangle's centroid and compare it to the planar interpolation of the triangle's vertices. The distance or normal difference can serve as error. This automatically accounts for both radial pattern and vertical curvature. In practice, the current shader's `compute_style_variation` is a proxy for radial curvature. We should extend `compute_importance` to include something like:

```
// pseudo-WGSL
let z = t * H;
// sample surface at triangle mid (theta_mid, t_mid)
let r_mid = compute_outer_radius(theta_mid, t_mid);
// estimate surface at mid via linear interp of vertices:
let r_lin = 0.25 * (r(theta0,t0)+ r(theta1,t0)+ r(theta0,t1)+
r(theta1,t1)); // corners
let curvature_err = abs(r_mid - r_lin);
base_imp += curvature_err * K;  // K some weight or scale factor
```

This captures vertical variation (because if base profile is curved, r_mid will deviate from linear average). Additionally, for inner/outer, we might compute normal differences. A simpler approach is to ensure that the initial mesh or the target quality accounts for vertical curvature by user input (e.g., if flare_exp is large, advise using higher n_z or enabling fine vertical refinement).

2. **Tune Min Triangle Size / Depth:** Set a reasonable floor like min edge length ~ 0.1 mm (assuming units mm) – meaning don't refine beyond what a printer can resolve. The `minQuadSize` uniform (chunk4.y) is intended for this. Currently it's probably very small or zero. We should set it to correspond to e.g. (2π/n_theta_initial * H/n_z_initial) / 4^maxDepth. Essentially, decide a maxDepth (like 6) or a min param segment length. This prevents endless tiny splits. For instance, if after 5 subdivisions a quad's size in θ or t is < 1e-3 (0.1% of domain), that might be enough.

In summary, the proposed architecture is:

- **Preprocess:** Use feature extraction results to embed feature lines in the mesh topology (via CDT or at least splitting base mesh along feature θ values).
- **Adaptive Loop:** Refine triangles with a global view – either until targetTriangles reached or until max error < tolerance, whichever comes first.
- **Error Metric:** Combine radial (pattern) and axial (profile) curvature. Ensure sharp edges (features) yield infinite or very high error if not already edges – so they always get refined or preserved.
- **Stop Conditions:** Use targetTriangles as a hard limit. Use min triangle size or maxDepth as secondary safety.

This architecture is more robust: it guarantees key features exactly, distributes triangles efficiently, and gives the user predictable control over output size. The trade-off is added complexity (CDT and more CPU involvement). However, the mesh sizes we target (<<1M) are manageable for modern CPUs in Javascript for CDT (especially with a web worker). Alternatively, some parts of this can be GPU-accelerated (but CDT is easier on CPU).

To illustrate the new approach, here's a **stage plan**:

1. **Feature-Constrained Base Mesh Generation:** Compute feature lines (CPU post-process of shader output). Insert those lines into base mesh. (E.g., start with coarse meshBuilder output, then add vertices where its faces intersect feature lines and split those faces.) This yields a constrained base mesh without drastically increasing polygon count.

2. **Adaptive Refinement with Budget:** Use the triangle-subdivision GPU loop but in controlled increments. After each subdivision pass, read back the triangle count (already done) and possibly error metrics (we could encode the max error found in an atomic or so). Decide whether to continue. Alternatively, perform adaptive refinement purely on CPU for final export (since after constraint insertion, CPU could subdivide ~few hundred k triangles in a second or two using efficient data structures – this is a consideration if GPU complexity grows).
3. **Vertex Evaluation:** Same as current – compute final coordinates for all vertices. Since we kept param coordinates accurate (especially on feature edges, vertices lie exactly on the analytic ridge/valley), the resulting surface matches the design closely.
4. **Output Export:** Use binary STL export (already planned/implemented – e.g., the new core might have `write_stl_binary`). Also consider offering 3MF for smaller files and including color per feature perhaps to visualize ridges (just an idea, not required).

Testing & Validation Plan

A comprehensive testing strategy is crucial to ensure the fixes work and no regressions are introduced:

- **Unit Tests on Geometry Calculations:** Test the base radius and style functions for correctness. For example, ensure `baseRadius(z)` matches the Python reference (including logistic flare behavior) by comparing a sample of outputs [1] [11]. Test combined `compute_outer_radius` + `compute_twist` against CPU `build_pot_mesh` results to confirm the GPU formula reproduces the same vertex positions for a given θ,z (within floating error).

- **Analytic Surface Deviation Tests:** We can create a utility to compute the max deviation between the mesh and the true surface. For a given pot design, sample 1000 random (θ,z) points, compute actual coordinates via the formula, then find the closest triangle on the mesh and distance. Before fixes, we expect large deviations in some cases (especially for ridges if not captured). After implementing feature constraints, those deviations should drop near zero on ridges. We automate this check for a few designs:

- A simple style (no pattern) but high flare – check that the max deviation decreases when we increase `n_z` or when adaptive is enabled. This ensures smooth curvature handled.

- A patterned style (e.g., SpiralRidges) – ensure that ridge peak error is below a small threshold (meaning the ridge is explicitly in mesh). Without feature edge, the error would be on order of wall thickness if ridge was approximated; with feature edge, error is just the difference between a sharp crease and how we represent it (which should be zero if it's truly an edge).

- **Mesh Quality Metrics:** Employ mesh quality criteria to catch issues:

- **Manifoldness & Watertightness:** Verify each edge is shared by exactly 2 faces (except outer boundary of drain hole) – no missing facets or holes. Our construction method should guarantee this, but we test on a variety of parameter extremes (very thin wall, large drain) to ensure clamp logic didn't leave gaps. Also confirm no duplicate vertices or stray unreferenced vertices.
- **Normals & Flips:** Compute normals for each face and ensure consistency (adjacent faces on smooth surfaces have normals within expected angle, faces on opposite sides of a sharp ridge have a clear normal jump which is intended). We also ensure there are no inward-facing normals on outer

surfaces. One way: take random points on the mesh and evaluate if they lie inside or outside the solid defined by the mesh. Since this is a solid of revolution type shape, an outside point (far away) should register outside by a winding test. Any abnormal normal orientation could indicate a flipped triangle – we should catch those if any appear (none expected if we keep winding consistent – e.g., outer surface wound CCW, inner CW, etc., as in geometry.py).

- **Seam continuity:** Programmatically check that the first column of vertices (θ=0) and last column (θ ~2π) are identical (within tolerance) in XYZ. Also check that no "split" occurred at seam – i.e., count of seam edges equals n_z (meaning each layer connects across the seam exactly once). Any mismatch would signal a seam gap.

- **Performance Tests:** Measure the triangle count and export time for a few scenarios:

- Default pot (no pattern) at Standard quality -> ensure triangle count is reasonable (and ideally reduced from before if we avoid over-refining smooth surfaces).
- A complex pattern at high quality with a set target (e.g., 500k triangles) -> verify that the output adheres to the target (no huge overflow) and visually still captures the pattern. This tests the budget control.

- Edge case: style with extremely subtle features – e.g., a very low-amplitude ripple. If feature extraction fails or returns none, our modified approach should still handle it: it won't find constraints (since truly negligible features shouldn't matter), and the adaptive refine might decide not to refine much, which is fine. But we need to ensure it doesn't erroneously refine to max or something. So test with hr_petal_amp = 0 (flat wall) and ensure no refinement beyond base grid, output ~base triangles count. Then gradually increase amplitude and see that refinement kicks in appropriately when features become significant.

- **Regression Test for Known Failures:** Re-run the exact cases that produced the known issues:

- The scenario that yielded the 43M-triangle STL: after fixes, this should either respect a triangle cap or require user to explicitly allow such high detail. If we set targetTriangles to, say, 1M, we expect the output ~1M triangles and visually the pattern is still well represented. We'd compare the ridge definition before vs after – the after version should show the ridges as sharp edges (we can slice the mesh and measure profile).

- Cases where feature_extract returned zero: perhaps a design with a very smooth style. We ensure that even if no features, the mesh still refines enough to achieve general smoothness (and doesn't blow up unexpectedly). And if no features are expected (like a plain pot), that's fine – the mesh should then basically be the base mesh or adapt just for smooth flare if needed.

- **Print Simulation:** (If resources allow) Pass the output meshes into a slicer or a 3D print utility to ensure they slice without errors. Specifically, check that the inner wall smoothing indeed resolved the non-manifold "fold-over" issue: slicers (like Cura or PrusaSlicer) should report no errors. If we generate a pot with extreme pattern (like "WaveInterference" style which was mentioned to cause inner wall folds), the new pipeline should output a manifold mesh (we can also algorithmically check that inner surface radius never *decreases* with height, to avoid inverted triangles).

- **User Acceptance Visual Check:** Finally, manually inspect a few outputs:

- Do ridges look knife-edge sharp and correctly placed? (Compare against a high-res preview or against the mathematical expectation.)
- Is the triangle density higher near features and lower in smooth areas, as intended (no more all-43-million uniformly tiny)? Visually, one should see larger, clean triangles on flat areas and fine ones tracing the patterns.
- No lingering seam line or anomalies at symmetry boundaries.

By systematically validating these aspects, we can be confident the fixes address the root causes. In particular, the combination of constrained meshing for features and controlled refinement for smoothness and count will elevate PotFoundry's output quality to professional grade: **sharp where it should be, smooth where it should be, and efficient in triangle use**.

---

1 2 3 4 5 6 7 8 9 10 11 geometry.py

file://file-MWDrsCkX7uNi8xfDjSu8xv